PostgreSQL Secrets Management

Table of Contents

Learning Objectives

By the end of this module, you will be able to:

- Use HashiCorp Vault for PostgreSQL credential management
- Set up dynamic database credentials with automatic rotation
- Implement .pgpass and pg_service.conf for password-free operations
- Rotate SSL certificates without downtime
- Integrate AWS Secrets Manager with PostgreSQL applications
- Avoid common credential management pitfalls

The Secrets Problem

```
BAD PATTERNS (Never Do These):
┌─────────────────────────────────────────────────────┐
│ 1. Hardcoded in source code:                        │
│    conn = psycopg2.connect("password=hardcoded!")   │
│    → Visible in git history forever                 │
│                                                      │
│ 2. Stored in unencrypted config files:              │
│    DB_PASSWORD=mypassword  # in .env                │
│    → Any user who can read the file knows the pass  │
│                                                      │
│ 3. Passed as command-line arguments:                │
│    psql -U user -W password dbname                  │
│    → Visible in process list (ps aux)               │
│                                                      │
│ 4. Stored in shell history:                         │
│    export DB_PASSWORD=secret; psql...               │
│    → Visible in ~/.bash_history                     │
```

```
└──────────────────────────────────────────────────┘


GOOD PATTERNS:
┌──────────────────────────────────────────────────┐
│ 1. Vault/Secrets Manager: Fetch at runtime       │
│ 2. Environment variables: Inject at deploy time  │
│ 3. .pgpass: Encrypted or protected file          │
│ 4. Instance role/IAM: No credential at all       │
│ 5. SSL certificates: No passwords for services   │
└──────────────────────────────────────────────────┘
```

HashiCorp Vault Integration

Vault Setup for PostgreSQL

```
# Install Vault
curl -fsSL https://apt.releases.hashicorp.com/gpg | gpg --dearmor | \
    sudo tee /usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=...] https://apt.releases.hashicorp.com $(lsb_release -cs)
main" | \
    sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt-get update && sudo apt-get install -y vault

# Start dev server (testing only)
vault server -dev -dev-root-token-id="root"
export VAULT_ADDR='http://127.0.0.1:8200'
export VAULT_TOKEN='root'

# Enable the database secrets engine
vault secrets enable database

# Configure Vault to connect to PostgreSQL
vault write database/config/my-postgresql-database \
    plugin_name=postgresql-database-plugin \
    allowed_roles="*" \
    connection_url="postgresql://{{username}}:{{password}}@localhost:5432/mydb" \
    username="vault_admin" \
    password="VaultAdminPass!" \
    password_authentication=scram-sha-256
```

```
-- Create the Vault admin user in PostgreSQL
CREATE ROLE vault_admin WITH LOGIN CREATEROLE PASSWORD 'VaultAdminPass!';
GRANT ALL ON DATABASE mydb TO vault_admin;
```

Static Credentials with Vault

```
# Store existing PostgreSQL password in Vault
vault kv put secret/myapp/database \
```

```
    host="db.example.com" \
    port="5432" \
    database="mydb" \
    username="appuser" \
    password="AppPassword123!"

# Retrieve the secret
vault kv get secret/myapp/database
vault kv get -format=json secret/myapp/database | jq '.data.data'

# Application retrieves at startup:
DB_PASSWORD=$(vault kv get -field=password secret/myapp/database)
```

Dynamic Credentials (Vault creates and manages DB users)

```
# Create a Vault role that generates temporary DB credentials
vault write database/roles/app-role \
    db_name=my-postgresql-database \
    creation_statements="CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}'
VALID UNTIL '{{expiration}}';
                         GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN
SCHEMA public TO \"{{name}}\";
                         GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO \"
{{name}}\";" \
    default_ttl="1h" \
    max_ttl="24h"

# Application requests credentials:
vault read database/creds/app-role
# Returns:
# lease_id          database/creds/app-role/abc123
# lease_duration    1h
# username          v-app-role-xkcd1234
# password          A1b2C3d4E5f6G7h8

# These credentials auto-expire after 1 hour
# Vault automatically creates a new user each time
```

Application Integration

```
# Python application using Vault dynamic credentials

import hvac   # pip install hvac
import psycopg2
import time

class DatabasePool:
    def __init__(self, vault_addr, vault_token, role_name):
        self.vault_client = hvac.Client(url=vault_addr, token=vault_token)
```

```
        self.role_name = role_name
        self.credentials = None
        self.expiry = 0

    def get_credentials(self):
        """Get or refresh database credentials from Vault."""
        if time.time() > self.expiry - 60:  # Refresh 60s before expiry
            creds = self.vault_client.secrets.database.generate_credentials(
                name=self.role_name
            )
            self.credentials = {
                'username': creds['data']['username'],
                'password': creds['data']['password'],
                'expiry': time.time() + creds['lease_duration']
            }
            self.expiry = self.credentials['expiry']
        return self.credentials

    def connect(self):
        creds = self.get_credentials()
        return psycopg2.connect(
            host="db.example.com",
            port=5432,
            database="mydb",
            user=creds['username'],
            password=creds['password'],
            sslmode="verify-full",
            sslrootcert="/etc/ssl/pg/ca.crt"
        )

# Usage
pool = DatabasePool("http://vault.internal:8200", vault_token, "app-role")
conn = pool.connect()
```

Environment Variables Best Practices

```
# PGPASSWORD: Set PostgreSQL password via environment variable
# (Avoid for interactive shells — visible in process table and env dumps)

# Shell scripts (acceptable for automation)
export PGPASSWORD="SecurePassword123!"
psql -h db.example.com -U appuser -d mydb -c "SELECT version();"
unset PGPASSWORD  # Clear immediately after use

# Docker/container: inject at runtime (not in Dockerfile)
# docker run --env PGPASSWORD="..." myapp
# Or use Docker secrets / Kubernetes secrets

# PostgreSQL libpq environment variables
export PGHOST=db.example.com
```

```
export PGPORT=5432
export PGDATABASE=mydb
export PGUSER=appuser
export PGPASSWORD=SecurePassword123!
export PGSSLMODE=verify-full
export PGSSLROOTCERT=/etc/ssl/pg/ca.crt

# These are picked up automatically by psql, pg_dump, pg_basebackup, etc.
```

```
# Kubernetes: Mount secret as environment variable
# kubernetes/deployment.yaml
# env:
#   - name: PGPASSWORD
#     valueFrom:
#       secretKeyRef:
#         name: postgres-secret
#         key: password
```

.pgpass File

The `.pgpass` file stores PostgreSQL credentials for password-free connections.

```
# Location: ~/.pgpass (home directory of the user running psql)
# Format: hostname:port:database:username:password

# Create .pgpass
cat > ~/.pgpass << 'EOF'
# hostname:port:database:username:password
db.example.com:5432:mydb:appuser:SecurePassword123!
db.example.com:5432:postgres:postgres:SuperSecret!
192.168.1.100:5432:replication:replicator:ReplicaPass!

# Wildcards: * matches any value
*:5432:*:monitoring:MonitoringPass!
localhost:5432:*:postgres:LocalAdminPass!
EOF

# CRITICAL: .pgpass must be owned by user and not readable by others
chmod 600 ~/.pgpass

# Test: psql should not prompt for password
psql -h db.example.com -U appuser -d mydb

# For system-level use (cron jobs, etc.)
# Place in postgres user's home directory
sudo -u postgres bash -c 'echo "localhost:5432:*:postgres:pass" > ~/.pgpass && chmod
600 ~/.pgpass'
```

.pgpass for pg_basebackup

```
# /var/lib/postgresql/.pgpass (postgres user's home)
# For automated backups and replication
cat > /var/lib/postgresql/.pgpass << 'EOF'
# Replication
primary.internal:5432:replication:replicator:ReplicaPass123!
# Monitoring
*:5432:postgres:postgres:AdminPass123!
EOF
chmod 600 /var/lib/postgresql/.pgpass
chown postgres:postgres /var/lib/postgresql/.pgpass
```

Connection Service File

`pg_service.conf` allows named connection profiles — abstract away connection details.

```
# /etc/postgresql-common/pg_service.conf (system-wide)
# or ~/.pg_service.conf (per-user)

[prod-primary]
host=db.example.com
port=5432
dbname=mydb
user=appuser
sslmode=verify-full
sslrootcert=/etc/ssl/pg/ca.crt

[prod-replica]
host=replica.example.com
port=5432
dbname=mydb
user=readonly_user
sslmode=verify-full

[staging]
host=staging-db.example.com
port=5432
dbname=mydb_staging
user=appuser

[local-dev]
host=localhost
port=5432
dbname=mydb_dev
user=developer
sslmode=disable
```

```
# Connect using service name
psql service=prod-primary
psql service=prod-replica -c "SELECT COUNT(*) FROM orders;"
```

```
# In connection strings
PGSERVICE=prod-primary psql
# Or: psql "service=prod-primary"

# In Python
conn = psycopg2.connect("service=prod-primary password=fromenvvar")
```

AWS Secrets Manager

```
# Store PostgreSQL credentials in AWS Secrets Manager
aws secretsmanager create-secret \
    --name "prod/postgresql/appuser" \
    --description "Production PostgreSQL credentials for appuser" \
    --secret-string '{
        "host": "db.example.com",
        "port": 5432,
        "database": "mydb",
        "username": "appuser",
        "password": "SecurePassword123!",
        "engine": "postgres"
    }'

# Retrieve secret
aws secretsmanager get-secret-value \
    --secret-id "prod/postgresql/appuser" \
    --query SecretString \
    --output text | python3 -m json.tool
```

```
# Python: Fetch from Secrets Manager
import boto3
import json
import psycopg2

def get_db_credentials(secret_name):
    client = boto3.client('secretsmanager', region_name='us-east-1')
    response = client.get_secret_value(SecretId=secret_name)
    return json.loads(response['SecretString'])

# Usage (application uses IAM role, no hardcoded AWS credentials)
creds = get_db_credentials('prod/postgresql/appuser')
conn = psycopg2.connect(
    host=creds['host'],
    port=creds['port'],
    database=creds['database'],
    user=creds['username'],
    password=creds['password'],
```

```
    sslmode='verify-full'
)
```

```
# Enable automatic rotation (Secrets Manager rotates password every 30 days)
aws secretsmanager rotate-secret \
    --secret-id "prod/postgresql/appuser" \
    --rotation-rules AutomaticallyAfterDays=30 \
    --rotation-lambda-arn arn:aws:lambda:us-east-1:123456789:function:rotate-
postgres-secret
```

SSL Certificate Lifecycle

```
# Certificate expiration monitoring
check_cert_expiry() {
    CERT_FILE=$1
    DAYS_WARN=${2:-30}

    EXPIRY_DATE=$(openssl x509 -in "$CERT_FILE" -noout -enddate | cut -d= -f2)
    EXPIRY_EPOCH=$(date -d "$EXPIRY_DATE" +%s)
    CURRENT_EPOCH=$(date +%s)
    DAYS_LEFT=$(( (EXPIRY_EPOCH - CURRENT_EPOCH) / 86400 ))

    if [ $DAYS_LEFT -lt $DAYS_WARN ]; then
        echo "WARNING: Certificate $CERT_FILE expires in $DAYS_LEFT days
($EXPIRY_DATE)"
        return 1
    fi
    echo "OK: Certificate $CERT_FILE valid for $DAYS_LEFT days"
    return 0
}

check_cert_expiry /etc/postgresql/16/main/server.crt 30

# Add to cron:
# 0 8 * * * /usr/local/bin/check_cert_expiry /etc/postgresql/16/main/server.crt 30 |
mail -s "PG Cert Check" dba@company.com
```

Certificate Rotation (PostgreSQL 14+)

```
#!/bin/bash
# rotate_pg_ssl_cert.sh — Zero-downtime certificate rotation

PGDATA=/var/lib/postgresql/16/main
NEW_CERT=/tmp/new_server.crt
NEW_KEY=/tmp/new_server.key

echo "Rotating PostgreSQL SSL certificate..."
```

```
# Backup current
cp $PGDATA/server.crt $PGDATA/server.crt.bak.$(date +%Y%m%d)

# Deploy new certificate
cp $NEW_CERT $PGDATA/server.crt
# Note: key rotation requires restart; cert rotation does not

# Set permissions
chmod 644 $PGDATA/server.crt
chown postgres:postgres $PGDATA/server.crt

# Reload PostgreSQL (new connections use new cert, existing unaffected)
sudo -u postgres psql -c "SELECT pg_reload_conf();"

# Verify new cert is being served
sleep 2
openssl s_client -connect localhost:5432 -starttls postgres 2>/dev/null | \
    openssl x509 -noout -dates

echo "Certificate rotation complete."
```

Dynamic Database Credentials with Vault

```
# Full workflow: Vault creates short-lived DB users

# 1. Application requests credentials
DB_CREDS=$(vault read -format=json database/creds/app-role)
DB_USER=$(echo $DB_CREDS | jq -r '.data.username')
DB_PASS=$(echo $DB_CREDS | jq -r '.data.password')
LEASE_ID=$(echo $DB_CREDS | jq -r '.lease_id')

# 2. Connect to database
psql "host=db.example.com user=$DB_USER password=$DB_PASS dbname=mydb
sslmode=verify-full"

# 3. Renew before expiry (optional)
vault lease renew $LEASE_ID

# 4. Revoke when done (explicit cleanup, or auto-expires)
vault lease revoke $LEASE_ID

# The user is DELETED from PostgreSQL after revocation
```

Password Rotation Procedures

```
-- PostgreSQL role: routine password rotation

-- Step 1: Generate new password (application or external)
```

```
-- New password comes from Vault, random generator, or admin

-- Step 2: Change password in PostgreSQL
ALTER ROLE appuser WITH PASSWORD 'NewGeneratedPassword!';

-- Step 3: Update Vault/Secrets Manager with new password
-- (done by rotation script)

-- Step 4: Verify application can still connect with new password

-- Step 5: Remove old password from all caches/config files

-- Automated rotation with Vault database engine:
vault write database/rotate-role/my-postgresql-database/appuser
-- Vault changes the password in PostgreSQL and stores new value
```

```
#!/bin/bash
# rotate_db_password.sh

DB_HOST=db.example.com
DB_USER=appuser
VAULT_SECRET_PATH=secret/myapp/database

# 1. Generate new password
NEW_PASS=$(openssl rand -base64 32 | tr -d '+/=' | head -c 24)

# 2. Update PostgreSQL
PGPASSWORD=$OLD_PASS psql -h $DB_HOST -U $DB_USER -d postgres \
    -c "ALTER ROLE $DB_USER WITH PASSWORD '$NEW_PASS';"

# 3. Update Vault
vault kv patch $VAULT_SECRET_PATH password="$NEW_PASS"

# 4. Verify connection with new password
PGPASSWORD=$NEW_PASS psql -h $DB_HOST -U $DB_USER -d mydb -c "SELECT 1;" && \
    echo "Password rotation successful" || \
    echo "ERROR: Cannot connect with new password!"

# 5. Clear from environment
unset NEW_PASS OLD_PASS
```

Common Mistakes

1. **Committing passwords to git** — use `git-secrets` or similar pre-commit hooks
2. **Logging passwords** — don't log connection strings that include passwords
3. **Sharing credentials across environments** — dev credentials must not work in production
4. **Never rotating passwords** — stale credentials are a risk
5. **Using PGPASSWORD in scripts visible to ps aux** — use .pgpass instead
6. **Storing Vault root token** — root tokens should only be used for initial setup
7. **Long-lived service account passwords** — prefer dynamic credentials

8. **Not revoking old credentials after rotation** — overlapping valid credentials
9. **Weak password generation** — use cryptographic randomness, not predictable patterns
10. **SSL certificate expiration not monitored** — surprise outage

---

Best Practices

1. **Never hardcode credentials** — always externalize
2. **Use dynamic credentials** (Vault) where possible for zero-credential-at-rest
3. **Rotate passwords regularly** — at minimum: when an employee leaves, quarterly for service accounts
4. **Use SSL certificates** instead of passwords for service-to-service connections
5. **Implement expiration** for all credentials (`VALID UNTIL`)
6. **Monitor certificate expiration** 30+ days in advance
7. **Use .pgpass** for automated scripts instead of PGPASSWORD env var
8. **Use pg_service.conf** to abstract connection parameters from application code
9. **Log secret access** — who accessed what credentials, when (via Vault audit log)
10. **Test rotation procedures** — verify application reconnects after rotation

---

Interview Questions

Q1: What is the PGPASSWORD environment variable and what are its security concerns?

A: `PGPASSWORD` is an environment variable that libpq reads for the database password. Security concerns: (1) Environment variables may be visible to other processes via `/proc/<pid>/environ` on Linux. (2) They can be leaked in log output or error messages. (3) If set in shell config files (`.bashrc`), they persist. Better alternatives: `.pgpass` file (chmod 600) or `-W` flag (interactive prompt). In automation, inject via secrets manager at runtime and unset immediately.

Q2: What is HashiCorp Vault's dynamic secrets feature and how does it work with PostgreSQL?

A: Vault's database secrets engine can create temporary, unique database credentials on demand. Configuration: tell Vault how to connect to PostgreSQL with an admin user. Define roles with SQL templates for user creation. When an application requests credentials, Vault (1) creates a new PostgreSQL role with a random username and password, (2) grants specified permissions, (3) sets a TTL (e.g., 1 hour). After TTL, Vault drops the role. Applications always use fresh credentials; no long-lived passwords.

Q3: What is pg_service.conf and why is it useful?

A: `pg_service.conf` (or `~/.pg_service.conf`) defines named connection profiles. Applications connect using `service=profile_name` instead of specifying host/port/database/user. Benefits: (1) Abstract connection parameters from application code. (2) Change server without code changes — update the service file. (3) Different service names for different environments. (4) Don't include passwords in the service file — combine with `.pgpass` or environment variables.

Q4: How do you implement zero-downtime credential rotation for a live application?

A: (1) Add the new password to the database role (PostgreSQL allows multiple valid passwords in some schemes, or use a grace period). (2) Update the secrets store with the new password. (3) Trigger application rolling restart so new processes use new credentials. (4) Verify all connections are using new password. (5) Remove old password. With Vault dynamic credentials, rotation is automatic — applications always request new credentials before TTL expires.

Q5: What is a `.pgpass` file and when should you use it?

A: A `~/.pgpass` file (chmod 600) stores hostname:port:database:username:password entries. When psql or other libpq clients connect, they look up matching entries and use the stored password. Use for: (1) `pg_basebackup` automation. (2) Cron-scheduled `pg_dump`. (3) DBA scripts that need password-free automation. (4) Replication connections from standby setup scripts. Wildcards (`*`) can match any database or host.

Q6: How would you prevent credentials from being committed to git?

A: (1) Add `.env`, `*.pgpass`, `postgresql.conf` to `.gitignore`. (2) Use `git-secrets` or `detect-secrets` pre-commit hooks that scan for credential patterns. (3) Use environment variable injection (CI/CD injects secrets at runtime). (4) Use `.env.example` with placeholders — never the actual values. (5) Use HashiCorp Vault or AWS Secrets Manager so credentials aren't in config files at all.

Q7: How do SSL certificates improve security compared to passwords?

A: Certificates eliminate shared secrets entirely — authentication is based on asymmetric cryptography. Benefits: (1) No password to steal or brute-force. (2) Private key never leaves the client machine. (3) Server verifies client by checking certificate signature (only the CA can sign valid certs). (4) Easy revocation via CRL. (5) No rotation needed for the authentication mechanism itself (just renew the cert before expiry). Use for service-to-service connections, replication, and automated tools.

Exercises and Solutions

Exercise 1: Implement Vault Dynamic Credentials

```
#!/bin/bash
# Complete Vault PostgreSQL integration

# Configure Vault
vault secrets enable database

vault write database/config/production-pg \
    plugin_name=postgresql-database-plugin \
    allowed_roles="webapp" \
    connection_url="postgresql://vault_admin:{{password}}@db.example.com:5432/mydb?
sslmode=verify-full" \
    username="vault_admin" \
    password="VaultAdminSecret!" \
    max_open_connections=5

# Create role for web application
vault write database/roles/webapp \
    db_name=production-pg \
    creation_statements="
        CREATE ROLE \"{{name}}\" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL
'{{expiration}}';
        GRANT app_readwrite TO \"{{name}}\";
    " \
    revocation_statements="DROP ROLE IF EXISTS \"{{name}}\";" \
    default_ttl="1h" \
    max_ttl="4h"
```

```
# Application requests credentials
vault read database/creds/webapp
```

Exercise 2: Certificate Monitoring Script

```
#!/bin/bash
# cert_monitor.sh — Monitor all PostgreSQL SSL certs

CERTS=(
    "/etc/postgresql/16/main/server.crt"
    "/etc/postgresql/16/main/client.crt"
    "/etc/ssl/pg/ca.crt"
)

WARN_DAYS=30
CRITICAL_DAYS=7

for CERT in "${CERTS[@]}"; do
    if [ ! -f "$CERT" ]; then
        echo "MISSING: $CERT"
        continue
    fi

    EXPIRY=$(openssl x509 -in "$CERT" -noout -enddate 2>/dev/null | cut -d= -f2)
    DAYS=$(( ($(date -d "$EXPIRY" +%s) - $(date +%s)) / 86400 ))

    if [ $DAYS -lt $CRITICAL_DAYS ]; then
        echo "CRITICAL: $CERT expires in $DAYS days ($EXPIRY)"
    elif [ $DAYS -lt $WARN_DAYS ]; then
        echo "WARNING: $CERT expires in $DAYS days ($EXPIRY)"
    else
        echo "OK: $CERT expires in $DAYS days ($EXPIRY)"
    fi
done
```

Cross-References

- 04_ssl_tls.md — SSL certificate details
- 01_authentication_methods.md — Auth methods using managed credentials
- 06_auditing.md — Audit credential usage
- ../13_Replication_HA/02_streaming_replication.md — .pgpass for replication